



Writing and Using Modules in Terraform

Introduction

Imagine you are setting up cloud infrastructure for multiple projects.

- **How do you avoid writing the same Terraform code repeatedly?**
- **How do you make Terraform configurations modular and reusable?**
- **How do you ensure consistency across different environments?**

Terraform Modules solve these problems!

Section 1: Learn

What is a Module in Terraform?

A module in Terraform is a collection of Terraform configuration files that define reusable infrastructure components.

Terraform modules help to:

1. **Reuse Terraform configurations** instead of duplicating code.
2. **Organize infrastructure** into separate, logical components.
3. **Make Terraform code easier to maintain and scale.**

Why Do We Need Modules?

Without Modules	With Modules
Repetitive Terraform code	Reusable and modular code
Difficult to update infrastructure	Update once, apply everywhere
Hard to manage large configurations	Organized and scalable setup



Without Modules	With Modules
Higher risk of errors	Improves consistency and reduces mistakes

Key Benefits of Terraform Modules

1. **Encapsulation** – Group related resources together.
2. **Reusability** – Define once, use multiple times.
3. **Portability** – Works across multiple cloud providers.
4. **Scalability** – Easier to manage and scale infrastructure.

An Interesting Anecdote

A DevOps team at a tech company had to **manually create infrastructure for each project**. After switching to Terraform modules, they **reduced deployment time by 50% and improved consistency across environments!**

Section 2: Practice

Step-by-Step Guide to Writing and Using Terraform Modules

1. Creating a Simple Terraform Module

Step 1: Create a Module Directory

```
mkdir -p modules/ec2_instance  
cd modules/ec2_instance
```

Step 2: Define the Module Configuration

Create a file **main.tf** inside the **modules/ec2_instance** directory:



```
resource "aws_instance" "web_server" {  
  ami      = var.ami_id  
  instance_type = var.instance_type  
}
```

Create a **variables.tf** file to define input variables:

```
variable "ami_id" {  
  description = "The AMI ID for the EC2 instance"  
  type       = string  
}
```

```
variable "instance_type" {  
  description = "The type of EC2 instance"  
  type       = string  
  default    = "t2.micro"  
}
```

Create an **outputs.tf** file to define output values:

```
output "instance_id" {  
  description = "The ID of the EC2 instance"  
  value      = aws_instance.web_server.id  
}
```

Step 3: Use the Module in the Main Terraform File

In the root directory, create a **main.tf** file and reference the module:

```
provider "aws" {  
  region = "us-east-1"
```



```
}  
  
module "ec2_instance" {  
    source      = "./modules/ec2_instance"  
    ami_id      = "ami-12345678"  
    instance_type = "t2.micro"  
}
```

Run Terraform commands:

```
terraform init  
terraform apply
```

What happens here?

- Terraform **downloads the AWS provider**.
 - The module **creates an EC2 instance** using defined variables.
-

2. Using Multiple Modules

Terraform allows multiple modules to be used together.

Example: Creating both **EC2 and S3 resources** using separate modules.

```
module "ec2_instance" {  
    source      = "./modules/ec2_instance"  
    ami_id      = "ami-12345678"  
    instance_type = "t2.micro"  
}  
  
module "s3_bucket" {
```



```
source    = "./modules/s3_bucket"

bucket_name = "my-unique-bucket-name"
}
```

Run Terraform:

```
terraform apply
```

Why is this useful?

- Keeps configurations modular and reusable.
 - Allows independent management of different resources.
-

3. Using Remote Modules from Terraform Registry

Terraform provides a **public registry** where pre-built modules are available.

Example: Using an **official AWS VPC module**:

```
module "vpc" {

  source = "terraform-aws-modules/vpc/aws"

  version = "3.14.0"

  cidr    = "10.0.0.0/16"
}
```

Run Terraform:

```
terraform apply
```

Why is this important?

- Saves time by using community-tested modules.
 - Reduces development effort.
-



4. Passing Outputs Between Modules

Outputs from one module can be used in another module.

Step 1: Define an Output in the First Module

Inside `modules/network/outputs.tf`:

```
output "vpc_id" {  
    value = aws_vpc.main.id  
}
```

Step 2: Use the Output in Another Module

```
module "ec2_instance" {  
    source      = "../modules/ec2_instance"  
    ami_id      = "ami-12345678"  
    instance_type = "t2.micro"  
    vpc_id      = module.network.vpc_id  
}
```

Run Terraform:

```
terraform apply
```

Why is this useful?

- Allows smooth data transfer between modules.

Real-World Example

A cloud engineering team needs to:



1. Reuse infrastructure components across multiple projects.
2. Ensure all teams use the same best practices.
3. Easily update configurations without breaking existing setups.

What did they do?

1. Created Terraform modules for EC2, S3, and VPCs.
2. Used remote modules from Terraform Registry.
3. Passed outputs between modules for better automation.

Result? The team reduced duplication, improved scalability, and increased efficiency!

Section 3: Know More

Frequently Asked Questions

1. What is a module in Terraform?
 - A **module** is a reusable group of Terraform configurations.
 2. Why should I use Terraform modules?
 - Modules **reduce code duplication, improve reusability, and simplify infrastructure management.**
 3. Can I use modules from Terraform Registry?
 - Yes, Terraform Registry provides **pre-built modules for AWS, Azure, and Google Cloud.**
 4. How do I pass variables to a module?
 - Use **variable** blocks inside the module and pass values in **main.tf**.
 5. Where can I practice Terraform modules?
 - Use **AWS Free Tier** and experiment with different modules.
-



Conclusion

Terraform **Modules** simplify infrastructure deployment by making **configurations reusable, scalable, and organized.**

Start by **writing simple modules, using multiple modules, and exploring Terraform Registry,** and soon you'll be **writing efficient and modular Terraform code!**